

15. Стек, опашка, дек, опашка с приоритети – обектно-ориентирана реализация на C++ на стек и опашка чрез масив.

1. Въведение

В основата на темата стои понятието **Абстрактен тип данни** (ADT). ADT е математически модел на структура от данни, който се описва чрез множество от стойности и множество операции над тях, без да се уточнява реализацията.

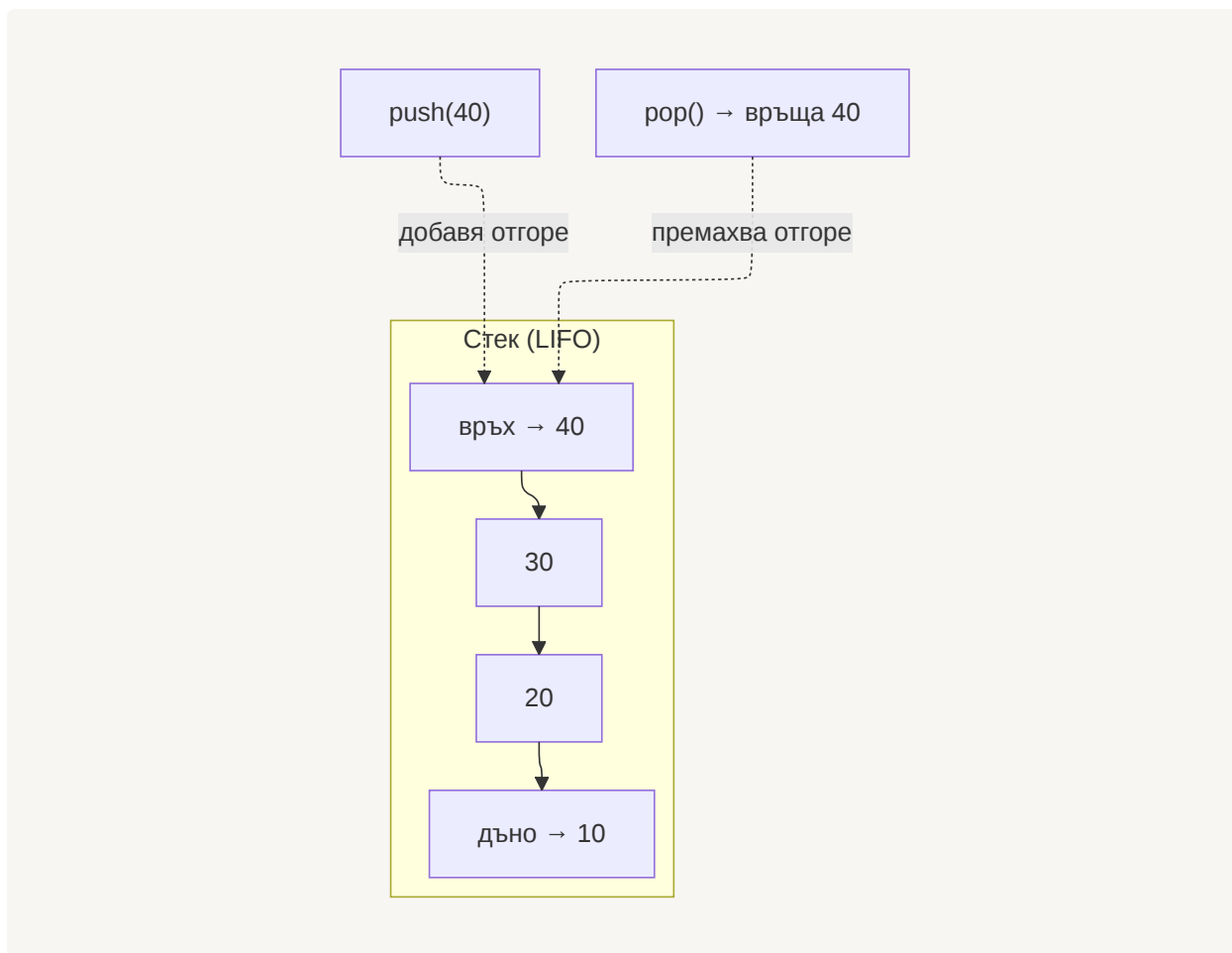
Стекът, опашката, декът и опашката с приоритети са **линейни структури от данни**: елементите им са подредени в редица, в която всеки елемент (с изключение на крайните) има точно един предходен и точно един следващ. Това, което ги отличава една от друга, е достъпът - правилото, определящо кой елемент може да бъде премахнат или прочетен във всеки момент.

Двете основни реализации са чрез **масив** (непрекъснатата памет) и чрез **свързан списък** (възли с указатели).

2. Стек (Stack)

2.1 Дефиниция

Стекът е линейна структура от данни, която работи на принципа LIFO (Last In, First Out). Добавянето и премахването на елементи **става само от единия край**, наречен **върх** (top). Класическата аналогия е купчина чинии: поставяме нова чиния отгоре и взимаме отново най-горната.



2.2 Основни операции

Стекът поддържа следните операции:

- `push(element)` - Добавя елемент на върха
- `pop()` - Премахва елемента от върха
- `top()` - Връща (без да го премахва) елемент на върха
- `empty()` - Връща true, ако стекът е празен
- `size()` - връща броя на елементите

Опитът за `pop()` или `top()` върху празен стек е грешка, наречена **underflow** (изпразване), а опитът за `push()` при пълен масив е **overflow** (препълване).

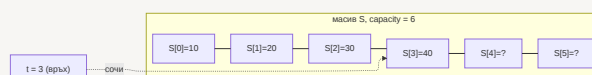
2.3 Приложения

Стекът се среща навсякъде в програмирането:

- Проверка на балансирани скоби и валидиране на синтаксис
- Стек на извикванията (call stack)
- Функцията "Undo" в текстовите редактори

2.4 Обектно-ориентирана реализация на стек чрез масив

Идеята е елементите да се пазят в масив `S`, а отделна целочислена променлива `t` да сочи **индекса на текущия връх**. Празен стек се обозначава с `t = -1`. При `push` увеличаваме `t` и записваме на новата позиция; при `pop` връщаме елемента и намаляваме `t`.



```
class ArrayStack {
private:
    int capacity;
    int* S;
    int t;

public:
    ArrayStack(int cap = 100) : capacity(cap), S(new int[cap]), t(-1) {}
    ~ArrayStack() { delete[] S; }

    int size() const {
        return t + 1;
    }

    bool empty() const {
        return t < 0;
    }
}
```

```

const int& top() const {
    if(empty()) {
        cout << "Празно е!" << endl;
    }

    return S[t];
}

void push(const int& el) {
    if (size() == capacity) {
        cout << "Пълно е!" << endl;
    }

    t++; // индексът на върха се променя (инкрементира с
e)
    S[t] = el;
}

void pop() {
    if (empty()) {
        cout << "Празно е!" << endl;
    }
    t--;
}
};

```

2.5 Анализ на сложността

Операция	Време
push , pop , top , empty , size	$O(1)$
Заета памет	$O(N)$, където N е капацитетът

Всички операции работят за константно време, защото действат само върху върха и не изискват обхождане. Основният недостатък на реализацията чрез

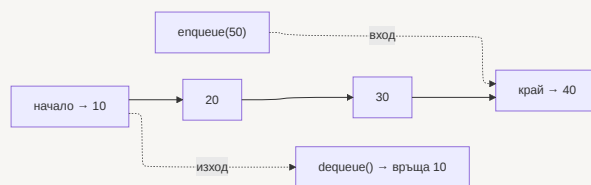
масив с фиксиран капацитет е, че трябва предварително да изберем максимален размер: твърде малък води до overflow, твърде голям - до загуба на памет. Решението е динамично нарастващ масив (при запълване се заделя нов масив с удвоен капацитет и елементите се копират). Тогава отделният push може да отнеме $O(N)$, но амортизираната сложност остава $O(1)$.

3. Опашка (Queue)

3.1 Дефиниция

Опашката е линейна структура от данни, работеща на принципа FIFO (Firs In, First Out).

За разлика от стека, тук добавянето и премахването стават от два различни края: нови елементи влизат в края, а се обслужват от началото. Аналогията е опашка от хора пред каса - обслужва се първият дошъл.



3.2 Основни операции

- `enqueue(e)` - Добавя елемент `e` в края на опашката
- `dequeue()` - Премахва елемент от началото
- `front()` - Връща (без да го премахва) елемента в началото
- `empty()` - Връща `true`, ако опашката е празна
- `size()` - Връща броя на елементите

3.3 Приложения

- Планиране на задачи в ОС

- Буфери при обмен на данни между процеси
- Обслужване на заявки в сървъри по реда на пристигането им

3.4 Реализация чрез масив

Решението тук е кръгов (цикличен) масив - индексите се завъртат обратно към началото с операцията остатък по модул (`% capacity`). Така масивът се използва като пръстен.

За да разграничим празна от пълна опашка, пазим броя на елементите `n` явно. Опашката е празна при `n == 0` и пълна при `n == capacity`.

3.5 Обектно-ориентирана реализация на опашка чрез кръгов масив

```
class ArrayQueue {
private:
    int capacity;
    int* Q;
    int f;
    int n;

public:
    ArrayQueue(int cap = 100) : capacity(cap), Q(new int[cap]), f(0), n(0) {}
    ~ArrayQueue() { delete[] Q; }

    int size() const {
        return n;
    }

    int empty() const {
        return n == 0;
    }

    const int& front() const {
```

```

        if(empty()) {
            cout << "Празно е!" << endl;
        }

        return Q[f];
    }

    void enqueue(const int& el) {
        if (n == capacity) {
            cout << "Пълно е" << endl;
        }

        int r = (f + n) % capacity;
        Q[r] = e;
        n++;
    }

    void dequeue() {
        if(empty()) {
            cout << "Празно е!" << endl;
        }

        f = (f + 1) % capacity;
        n--;
    }
};

```

3.6 Анализ на сложността

Операция	Време
<code>enqueue</code> , <code>dequeue</code> , <code>front</code> , <code>empty</code> , <code>size</code>	O(1)
Заета памет	O(N)

Благодарение на кръговото изчисление всички операции са O(1). Без него `dequeue` с местене би бил O(N).

4. Дек

4.1 Дефиниция

Декът е структура от данни, която позволява добавянето и премахването от двата края. Той е по-обща структура: със съответните ограничения декът може да се държи и като стек, и като опашка.

4.2 Основни операции

- `insertFront(e)` - Добавя елемент в началото
- `insertBack(e)` - Добавя елемент в края
- `removeFront()` - Премахва елемент от началото
- `removeBack()` - Премахва елемент от края
- `front()` / `back()` - Връща елемента в началото/края
- `empty()` - Проверка дали е празен
- `size()` - Връща брой елементи

4.3 Реализация и сложност

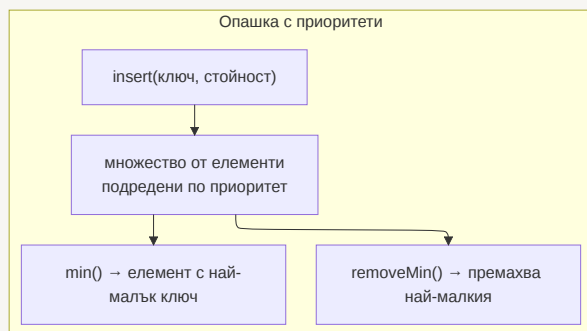
Декът се реализира ефективно чрез **кръгов масив** (както опашката, но с операции и от двата края) или чрез **двусвързан списък** (всеки възел има указатели към предходен и следващ). При двата подхода всички операции са $O(1)$.

Операция	Време
Всички операции (<code>insertFront</code> , <code>insertBack</code> , <code>removeFront</code> , <code>removeBack</code> , <code>front</code> , <code>back</code>)	$O(1)$

5. Опашка с приоритети

5.1 Дефиниция

При обикновената опашка редът на обслужване се определя от реда на постъпване (FIFO). При **опашката с приоритети** всеки елемент носи ключ за приоритет, а обслужва се винаги елементът с **най-висок приоритет**.



5.2 Основни операции

- `insert(e)` - Добавя елемент с даден ключ
- `min()` - Връща елемента с най-малък ключ (без премахване)
- `removeMin()` - Премахва елемента с най-малък ключ
- `empty()` , `size()` - Брой и проверка за празнота

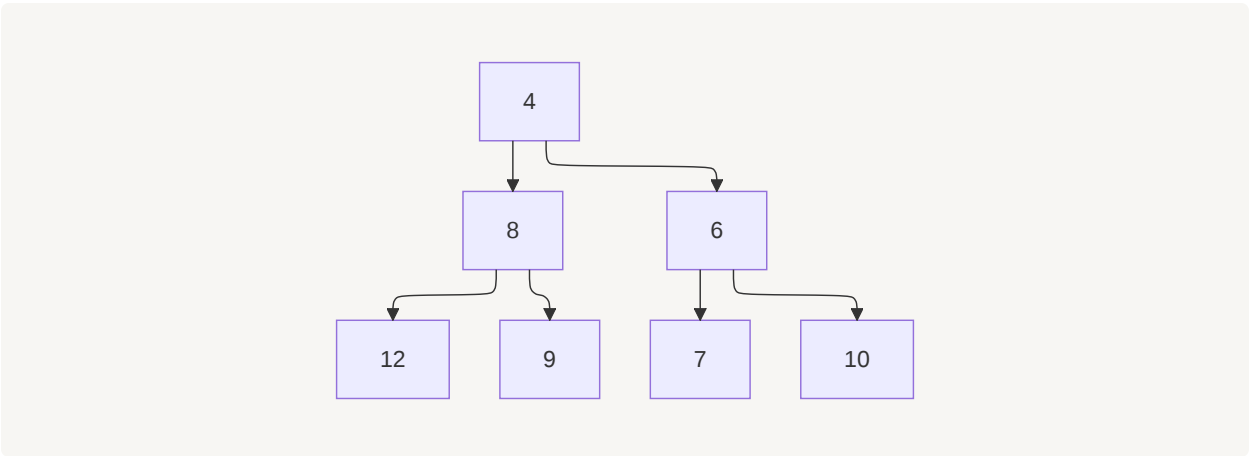
По същия начин се дефинира и опашка с приоритети по максимум (`max()` и `removeMax()`)

5.3 Реализация и сложност

Опашката с приоритети има няколко начина на реализация с различен компромис между скоростта на вмъкване и на изваждане:

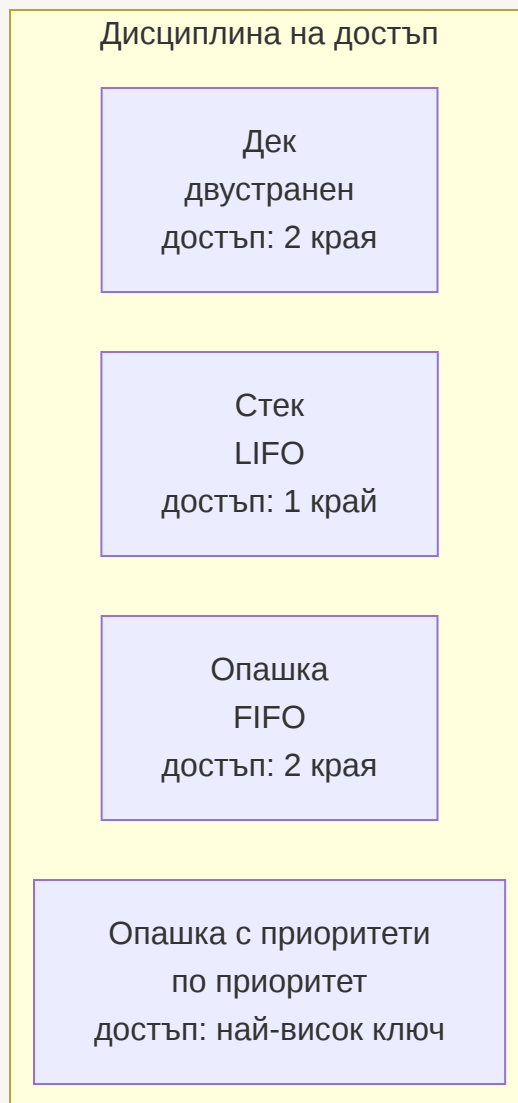
1. **Неподреден списък/масив.** `insert` просто добавя в края → **$O(1)$** , но `min` / `removeMin` трябва да обхождат всичко, за да намерят минимума → **$O(N)$** .
2. **Подреден списък/масив.** Елементите се пазят сортирани, затова `min` / `removeMin` са **$O(1)$** , но `insert` трябва да намери мястото → **$O(N)$** .
3. **Балансирано двоично дърво**, удовлетворяващо свойството на heap-a: ключът на всеки родител е $\leq$ ключовете на децата му (за min-heap). Това

дава $O(\log N)$ за `insert` и `removeMin` и $O(1)$ за `min`. Това е предпочитаната реализация.



Реализация	insert	min	removeMin
Неподреден списък	$O(1)$	$O(N)$	$O(N)$
Подреден списък	$O(N)$	$O(1)$	$O(1)$
Heap	$O(\log N)$	$O(1)$	$O(\log N)$

6. Сравнение



Структура	Принцип	Къде се добавя	Откъде се премахва	Сложност на осн. операции
Стек	LIFO	Върх	От върха	$O(1)$
Опашка	FIFO	В началото	От края	$O(1)$
Дек (Double-ended queue)	Двустраниен	От двете страни	От двете страни	$O(1)$

Опашка с приоритети	По приоритет	Произволно	Елемента с екстремален ключ	$O(\log N)$ за insert/removeMin и $O(1)$ за min
---------------------	--------------	------------	-----------------------------	---